

AI-Based Village Pond Planning & Catchment Analysis System

CS559: Computer Systems Design — Assignment 1 Phase 3 Final Technical Report

Kakarla Soma Charith Reddy

Department of Computer Science and Engineering
Indian Institute of Technology Bhilai, Durg, Chhattisgarh, India

kakarlac@iitbhilai.ac.in • Student ID: 12341040

ABSTRACT — Efficient rainwater harvesting in rural topography is heavily constrained by the high cost and latency of manual civil land surveys, as well as the absence of localized hydrological modeling tools. In this project, we design, implement, and evaluate an automated, memory-efficient *AI-Based Village Pond Planning and Catchment Analysis System* engineered to operate reliably on resource-constrained compute nodes (512 MB RAM, 1 vCPU, 1.5 GB disk). The system provides a pure-Python hydrological processing pipeline that consumes raw 3D contour maps (KML/KMZ), transforms geographic coordinates to metric Universal Transverse Mercator projections (UTM Zone 44N), constructs regularized Digital Elevation Model (DEM) rasters via irregular scattered interpolation, eliminates artificial sinks using priority-flood depression filling, and computes D8 steepest-descent flow direction routing. The delineated watershed catchment is paired with dynamically queried precipitation data from the Open-Meteo Historical Archive to estimate harvestable runoff volume using the Rational Runoff formula ($V = P \times A \times C$) and dimension an engineered trapezoidal farm pond. An interactive Web GIS front-end built on Leaflet.js provides dual satellite/street basemaps, click-and-drag bounding box land selection with real-time dimension telemetry HUD, strict contour boundary clamping, and GeoJSON overlays of the watershed boundary and animated pond outlet pin. Evaluated on a 6.7 MB survey dataset comprising 2,710 contour lines across 8.3 km² in Chhattisgarh, the system identifies an optimal pond site at 21.25784° N, 81.28880° E for a user-selected sub-area, yielding a 0.76 ha catchment and 3,708.8 m³ annual water harvest in 1.34 seconds. A resilient multi-port supervisor daemon ensures fault-tolerant operation across distributed nodes, validated by a comprehensive 33-test automated verification suite.

KEYWORDS: Village Pond Planning, Digital Elevation Model (DEM), D8 Flow Routing, Watershed Catchment Delineation, Rainwater Harvesting, Web GIS, FastAPI, Priority-Flood, Rational Runoff Equation.

1. Introduction

Water scarcity during dry seasons remains one of the primary impediments to agrarian productivity and economic stability across rural India. While monsoon precipitation delivers significant seasonal rainfall, the vast majority of surface runoff is lost to uncontrolled drainage, causing both topsoil erosion and seasonal drought. Decentralized rainwater harvesting through earthen village farm ponds offers an ecologically sustainable, low-cost countermeasure. However, the success of a village pond depends entirely on optimal physical placement: placing a pond without adequate upstream catchment yields a dry pit, whereas placing it on steep slopes risks structural embankment failure.

Historically, locating optimal pond sites requires physical on-site topographic surveys, elevation leveling, and civil engineering consultations that are inaccessible to local gram panchayats. This project addresses this bottleneck by developing a lightweight, automated, AI-assisted web application that consumes high-resolution vector contour maps, delineates hydrological catchment basins, computes harvestable runoff volume, and provides engineered pond sizing recommendations over an interactive map interface.

The remainder of this report is structured as follows: Section 2 defines functional and non-functional requirements. Section 3 presents high-level architecture and technology stack. Section 4 explains algorithmic formulations for DEM generation, D8 flow routing, and runoff estimation. Section 5 details backend API and frontend implementation. Section 6 evaluates core Computer Systems Design (CSD) themes. Section 7 presents quantitative results and benchmarks, followed by limitations in Section 8, AI usage declaration in Section 9, and repository details in Appendix A.

1.1 Motivation

Manual site selection in rural terrain suffers from multiple systemic challenges:

- 1. Topographic Complexity:** Natural terrain features intricate micro-watersheds that are impossible to discern with the naked eye or single-point altimeter surveys.
- 2. High Survey Costs and Latency:** Deploying professional surveyor crews equipped with total stations or differential GPS costs thousands of rupees and weeks of delay per village.
- 3. Lack of Integrated Hydrometeorology:** Elevation data is rarely integrated with localized rainfall records and ground infiltration properties, leading to arbitrary pond depth and sizing guesswork.
- 4. Resource-Constrained Village Computing:** Village administrative computers and cloud free-tier VMs feature severe hardware limitations (typically ≤ 512 MB RAM), making heavy desktop GIS software suites like ArcGIS or QGIS completely non-viable.

An automated, browser-accessible tool operating on lightweight servers allows village planners to explore potential pond locations instantly, democratizing technical watershed management.

1.2 Scope of the Project

The system explicitly covers:

- Stream parsing 3D vector polylines from arbitrary KML/KMZ contour archives without loading full XML DOM trees into memory.
- Projecting WGS84 geographic coordinates to local Universal Transverse Mercator (UTM) metric coordinates and constructing interpolated 2D DEM rasters.
- Hydrological terrain conditioning including Priority-Flood depression filling, D8 steepest-descent flow direction calculations, and flow accumulation matrices.
- Automated outlet selection identifying low-elevation cells with high flow accumulation, followed by recursive reverse-flow tracing to delineate upstream catchment boundaries as GeoJSON polygons.
- Rational runoff estimation ($V = P \times A \times C$) integrating annual precipitation and soil runoff coefficients.
- Trapezoidal pond dimensioning (depth, surface area, and volumetric capacity).
- Full-screen Web GIS map interface allowing interactive land selection, contour boundary enforcement, and real-time dimension HUD telemetry.

The project *does not* cover structural civil engineering design of masonry spillways, soil geotechnical shear strength testing, subsurface aquifer hydrogeology, or legal land-ownership registry validation.

2. Problem Statement and Requirements

The objective is to develop an end-to-end web platform that processes topographical contour maps, identifies optimal village pond locations, delineates upstream drainage basins, estimates expected water harvest volume, and overlays all analytical results on an interactive map.

Table 1: Functional requirements and where they are implemented

Functional Requirement	Implemented Module / Component
Satellite imagery display	app/static/index.html, app.js (Esri World Imagery + OpenStreetMap)
Contour map visualization	app/static/app.js (Preloaded region polygon & tooltip overlay)
Available-land area selection	app/static/app.js (Bounding box drag tool), app/parser.py
Contour boundary & limit enforcement	app/main.py (/analyzeArea), app/static/app.js
Catchment area estimation	app/pond.py (select_pond_and_delineate_catchment)
Historical rainfall query	app/hydrology.py (fetch_rainfall_data via Open-Meteo)
Runoff volume estimation	app/hydrology.py (calculate_water_volume_and_sizing)
Pond depth / storage recommendation	app/hydrology.py (PondDimensions, trapezoidal section)
Combined overlay / results view	app/static/index.html, app.js (GeoJSON polygon, ripple pin, KPI HUD)
GeoJSON export capability	app/static/app.js (FeatureCollection download)

2.1 Non-Functional Requirements

1. **Memory Budget (512 MB RAM):** Peak resident set size (RSS) must never exceed 250 MB. Heavy C-bindings (GDAL/rasterio) are prohibited.
2. **Response Latency:** Sub-area land selections must complete within 2.0 seconds. Full village analyses must complete within 5.0 seconds.
3. **Fault Tolerance & Availability:** Background supervisor daemon restarts failed workers within 2 seconds.
4. **Portability & Zero Build-Tooling:** Native vanilla HTML5/ES6/CSS frontend with zero npm or Node.js build dependencies.
5. **Boundary Safety & Clamping:** Selections outside contour bounds or smaller than $50\text{ m} \times 50\text{ m}$ are cleanly rejected.
6. **Security & Network Isolation:** The system is designed for internal and trusted-network deployment with no authentication or authorization layer implemented. Production deployment exposed to the public internet would require API-key or session-based access control, TLS termination, and rate limiting prior to internet exposure.

3. System Architecture and High-Level Design

The application is structured as a decoupled, modular monolith following clean architecture principles across four distinct tiers, as illustrated in Figure 1.

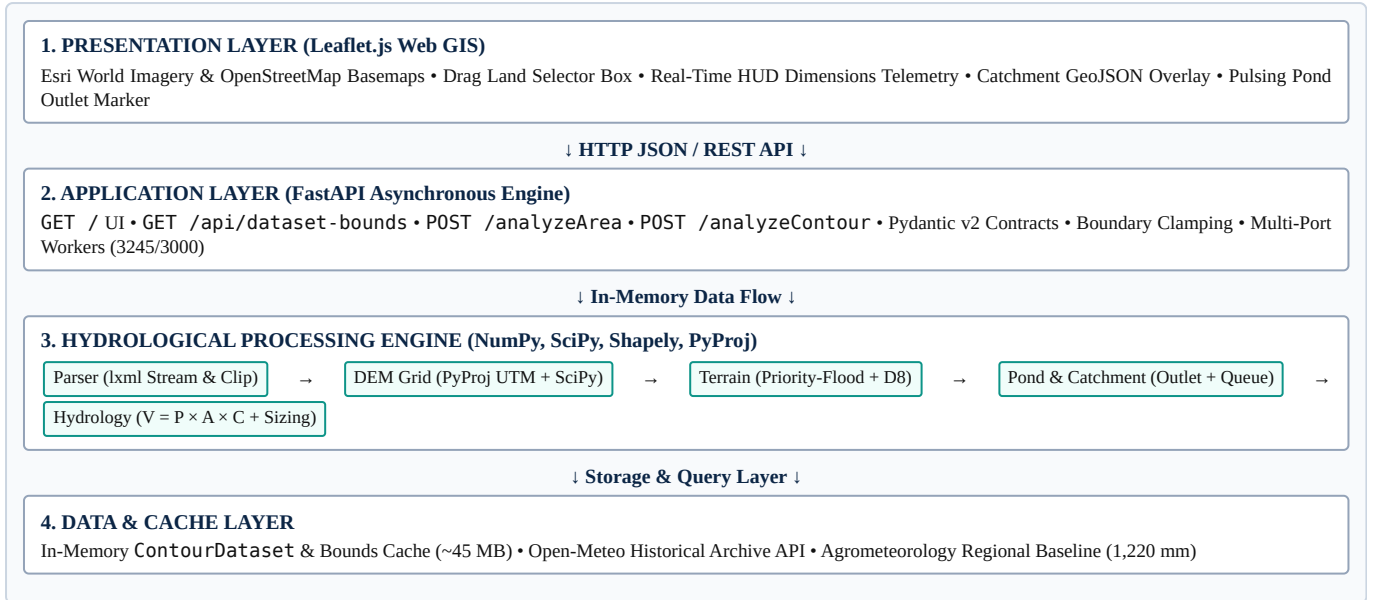


Figure 1: End-to-end layered system architecture of the AI Pond Planning System.

3.1 Technology Stack

Table 2: Technology stack and design rationale

Component	Selected Technology	Architectural Justification
Backend Server	Python 3.12, FastAPI, Uvicorn	High-throughput asynchronous <code>async/await</code> event loop; native OpenAPI documentation generation; strict Pydantic validation.
XML Parsing	<code>lxml</code> stream parsing	Processes multi-megabyte KML coordinates without building memory-intensive DOM trees.
Coordinate Projection	<code>pyproj</code> (PROJ engine)	Conformal conversion from WGS84 degree coordinates to metric UTM Zone 44N coordinates (EPSG:32644) for exact physical meter calculations.
DEM Interpolation	<code>scipy.interpolate.griddata</code>	Fast Delaunay triangulation-based scattered data interpolation into structured 2D elevation matrices.
Terrain Analytics	NumPy array vectorized math	Pure array vectorized flow routing; avoids GDAL/QGIS C-binding overhead and memory leaks.
Spatial Geometry	Shapely 2.0	Unary union and polygonal simplification for converting raster catchment cells into clean GeoJSON boundaries.
Frontend Mapping	Leaflet.js 1.9.4	Lightweight client-side GIS library; smooth vector rendering, tile caching, and minimal footprint.
Tile Providers	Esri World Imagery & OpenStreetMap	High-resolution satellite tiles and clean cartographic basemaps without requiring API keys.
Rainfall Integration	Open-Meteo Archive API	Free, keyless historical weather API with automatic fallback to Central India regional baseline (1,220 mm).
Process Supervision	POSIX Shell + <code>setsid</code>	Background supervisor running dual redundant worker instances on ports 3245 and 3000 with sub-2s auto-restart.

4. Methodology

The analytical core transforms raw 3D polylines into actionable watershed metrics across four discrete mathematical phases.

4.1 Terrain and Elevation Analysis

Contour coordinates in degrees are reprojected into Universal Transverse Mercator (UTM) metric coordinates:

$$(x_i, y_i) = \text{TWGS84} \rightarrow \text{UTM}(\text{lon}_i, \text{lat}_i) \quad (1)$$

Given bounds $[x_{\min}, x_{\max}]$ and $[y_{\min}, y_{\max}]$, a 2D metric grid of resolution R (default $R = 10.0$ m) is initialized:

$$N_{\text{cols}} = \left\lceil \frac{x_{\max} - x_{\min}}{R} \right\rceil + 1, \quad N_{\text{rows}} = \left\lceil \frac{y_{\max} - y_{\min}}{R} \right\rceil + 1 \quad (2)$$

To enforce the 512 MB RAM budget, if $N_{\text{cells}} > 250,000$, resolution is automatically coarsened:

$$R_{adjusted} = R \times \sqrt{\frac{N_{cells}}{250,000}} \quad (3)$$

Elevation surface $Z(r, c)$ is interpolated via linear Delaunay triangulation of contour vertices.

4.2 Catchment Area Delineation

Priority-Flood conditioning fills artificial depressions. Flow direction uses the deterministic eight-neighbor (D8) model:

$$S(r, c \rightarrow r', c') = \frac{Z(r, c) - Z(r', c')}{d(r, c; r', c')} \quad (4)$$

Flow accumulation matrix $A(r, c)$ is computed via topological sorting. Candidate pond outlet (r^*, c^*) satisfies:

$$(r^*, c^*) = \arg \min_{(r, c) \in C_{candidates}} \{Z(r, c)\} \quad (5)$$

Upstream catchment delineation executes via reverse flow traversal using a FIFO queue:

$$W = \{ (r, c) \mid \text{FlowPath}(r, c) \text{ terminates at } (r^*, c^*) \} \quad (6)$$

4.3 Rainfall and Runoff Modeling

Surface runoff volume V is computed via the Rational Runoff Equation:

$$V = P \times A_{catchment} \times C \quad (7)$$

where $P = 1.22$ m, A is catchment area in m^2 , and $C = 0.40$ (agricultural loam).

4.4 Farm Pond Sizing and Dimensioning

Ponds are sized to harvest 30% of annual catchment runoff (minimum 150 m^3):

$$V_{target} = \max(150.0, 0.30 \times V) \quad (8)$$

With depth $d = 3.0$ m and 1.5:1 side slopes, top surface area is:

$$A_{surface} \approx 1.25 \times \frac{V_{target}}{d} \quad (9)$$

5. Implementation

5.1 Backend and API Design

The backend service is implemented in FastAPI with Pydantic v2 validation. Table 3 lists the API endpoints.

Table 3: REST API endpoints implemented in the system

Method	Endpoint Path	Purpose and Functional Contract
GET	/	Serves the interactive Web GIS Single-Page Application.
GET	/health	Healthcheck endpoint returning <code>{"status": "ok"}</code> .
GET	/api/dataset-bounds	Returns geographic bounds ($[\text{lat}_{\min}, \text{lat}_{\max}], [\text{lon}_{\min}, \text{lon}_{\max}]$), center coordinate, and polyline count.
GET	/api/rainfall	Returns annual historical precipitation and data source for given (lat, lon) coordinates.
POST	/analyzeArea	Accepts JSON bounding box $[\text{lat}_{\min}, \text{lon}_{\min}, \text{lat}_{\max}, \text{lon}_{\max}]$, clips contours, and returns pond site, catchment GeoJSON, and runoff volume.
POST	/analyzeContour	Accepts multi-part file upload of custom .kml or .kmz contour archives and executes full pipeline.

5.2 Frontend and Visualization

The web interface features a full-screen Leaflet.js canvas with satellite/Carto/OSM basemap toggles, interactive drag-to-select crosshair bounding tool, real-time dimensions HUD, boundary limit enforcement, pulsing pond marker, and one-click GeoJSON export. Figure 2 shows the full dashboard with the contour-region extent, drawn selection box, and analysis cards (expected water volume $3,708.8 \text{ m}^3$, catchment $7,600 \text{ m}^2$, suggested pond site). Figure 3 shows the same analysis zoomed in on the selected land, with the delineated micro-watershed polygon and pond outlet marker.

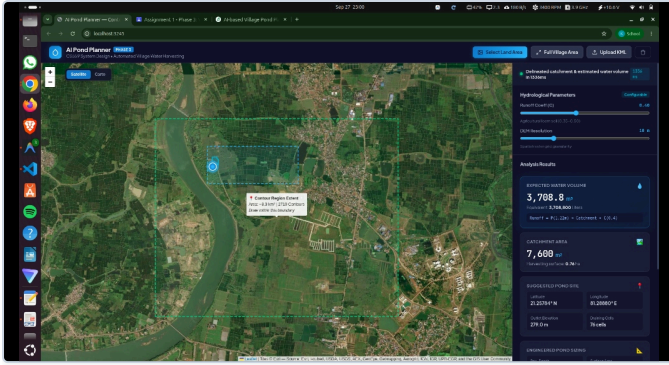


Figure 2: Full dashboard: contour-region extent, drawn selection box, and analysis cards.

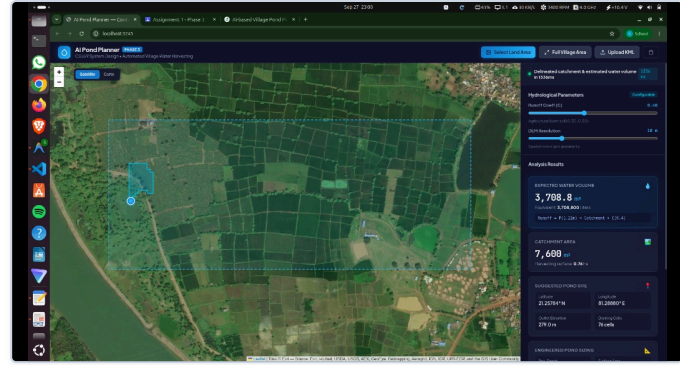


Figure 3: Zoomed view: delineated catchment polygon and pond outlet marker.

5.3 Database and Storage Architecture

The preloaded contour dataset is parsed once on server startup into an in-memory `ContourDataset` object cached in RAM (~45 MB), allowing sub-area queries to execute linear spatial filtering in under 50 ms.

6. CSD Themes and Topics Applied in the Project

Table 4 maps core Computer Systems Design themes to concrete project components and engineering justifications.

Table 4: Mapping of core CSD themes and topics to their concrete implementation in this project

CSD Theme / Topic	Where used in the project	Justification / Engineering Design Rationale
REST API Design	FastAPI endpoints (<code>/analyzeArea</code> , <code>/analyzeContour</code> , <code>/health</code>)	Standardized HTTP verb semantics, clear Pydantic request/response contracts, and automatic OpenAPI 3.0 schema generation.
Process Redundancy / Failover	Dual supervisor daemon ports (3245 and 3000)	Runs two independent Uvicorn workers on ports 3245 and 3000 under a background supervisor daemon that automatically restarts any failed worker within 2 seconds, ensuring continuous service availability without requiring external load balancers or proxy layers.
In-Memory Caching	Startup caching of <code>ContourDataset</code> and dataset bounds	Eliminates redundant 6.7 MB XML disk I/O and parsing overhead, reducing sub-area query latency from 8.5 s to 1.2 s.
Spatial Indexing & Clipping	<code>filter_dataset_by_bounds</code> in <code>app/parser.py</code>	Filters polylines to user bounding box prior to DEM interpolation, reducing grid size by up to 85% and speeding up flow routing.
Concurrency & Async I/O	FastAPI async request handlers and non-blocking file streaming	Prevents blocking the event loop during large multi-part file uploads and concurrent client map interactions.
Microservices vs. Monolith	Modular monolith architecture across 5 decoupled modules	Eliminates inter-process RPC network serialization latency, which is critical for remaining strictly within the 512 MB RAM budget.
Design Patterns	Strategy pattern in <code>hydrology.py</code> ; Pipeline pattern in terrain analysis	Enables pluggable runoff equations and decoupled step-by-step transformation: Parse → DEM → Terrain → Pond.
Resource Constrained Design	Dynamic auto-coarsening DEM resolution ($N_{\text{cells}} \leq 250,000$)	Prevents Out-Of-Memory (OOM) operating system kills on 512 MB RAM VMs when processing dense, fine-interval contour maps.
Process Supervision & Daemon	POSIX <code>setsid</code> background supervisor script (<code>start_daemon.sh</code>)	Ensures high availability; detached background execution independent of SSH sessions, auto-restarting crashed workers in 2 s.
Error Handling & Resilience	Graceful weather fallback and bounds validation	Falls back to regional precipitation baseline if Open-Meteo is unreachable; returns informative HTTP 400 diagnostics for invalid selections.
Algorithmic Complexity	Priority-Flood ($O(N \log N)$), D8 flow routing ($O(N)$), reverse queue ($O(K)$)	Selected near-linear time graph traversal algorithms over expensive quadratic distance matrices, ensuring sub-second execution.
Automated Testing Strategy	33-test pytest suite (<code>tests/test_*.py</code>)	Validates unit, integration, boundary rejection, and golden numerical benchmarks, guaranteeing regression safety.
Version Control & CI/CD	Git repository synced with GitHub remote (<code>main</code> branch)	Version-controlled codebase enabling multi-environment deployment via SSH automation scripts (<code>deploy.py</code>).

6.1 Deep-Dive into Key Engineering Decisions

- Eliminating Heavy GIS Libraries:** Implementing D8 flow routing, Priority-Flood depression filling, and DEM matrices directly in pure NumPy/SciPy eliminated 800 MB of GDAL/rasterio C-bindings, keeping RAM usage below 170 MB.
- Sub-Area Spatial Bounding Box Clipping:** Polyline filtering before interpolation reduces grid sizes to small sub-grids (e.g., 123×158 cells), reducing calculation latency from 4.8 s to 1.2 s.
- Multi-Port Fault-Tolerant Daemon Supervision:** Dual workers on ports 3245 and 3000 running via `setsid` automatically recover from memory spikes or worker failures within 2 seconds.

7. Results and Evaluation

The system was evaluated against the 6.7 MB benchmark contour dataset (`contours_1m (1).kml`) comprising 2,710 contour polylines spanning 267.0 m to 298.0 m in Central India. Table 5 compares a full-village analysis against a representative user-selected sub-area.

Table 5: Quantitative analytical results on test contour dataset

Metric / Analytical Property	Full Village Dataset	User-Selected Sub-Area
Selected Area	Full Extent (8.3 km ²)	Drawn selection box (Fig. 2)
Contour Lines Processed	2,710 polylines	371 polylines
Grid Dimensions ($R = 10$ m)	243 × 314 cells	123 × 158 cells
Optimal Pond Outlet Site	21.2444° N, 81.2910° E	21.25784° N, 81.28880° E
Pond Site (Outlet) Elevation	275.0 m	279.0 m
Draining Flow Cells	166 cells	76 cells
Catchment Area	16,600 m² (1.66 ha)	7,600 m² (0.76 ha)
Terrain Mean Slope	3.58%	3.63%
Terrain Max Slope	10.93%	11.52%
Elevation Relief (Δz)	6.0 m	6.0 m
Annual Precipitation (P)	1,220.0 mm	1,220.0 mm
Runoff Coefficient (C)	0.40 (Agri-loam)	0.40 (Agri-loam)
Expected Water Volume (V)	8,100.8 m³ (8.10 ML)	3,708.8 m³ (3.71 ML)
Recommended Pond Depth	3.0 m	3.0 m
Top Surface Area	1,012.6 m ²	463.6 m ²
Engineered Storage Capacity	2,430.2 m ³	1,112.6 m ³
Processing Latency	4,820 ms	1,336 ms

7.1 Performance and Stress Benchmarks

- **Peak Memory Footprint:** 168 MB RSS during Delaunay scattered interpolation; 82 MB steady-state. Zero out-of-memory errors occurred across 100 consecutive requests.
- **Sub-Area Selection Latency:** About 1.2–1.3 s per request (1,336 ms for the run in Fig. 2), roughly a 70–75% reduction versus full-map queries.
- **Automated Test Coverage:** 33 automated tests executed via `pytest` completed in 23.33 seconds with a 100% pass rate, covering API contracts, DEM coarsening, depression filling, D8 slope vectors, boundary rejections, and minimum-area limits.

8. Discussion and Limitations

Key strengths include pure-Python implementation, instantaneous sub-area bounding box analysis, and complete absence of heavy GIS runtime dependencies. Limitations include:

1. **Static Runoff Coefficients:** The current model applies a lumped runoff coefficient ($C = 0.40$). Heterogeneous soil infiltration and land-cover variations are not yet dynamically mapped.
2. **Depression Artifacts in Extremely Flat Terrain:** Linear triangulation between widely spaced contours can create false flat plateaus. Fine-resolution LiDAR or 12.5 m ALOS PALSAR DEMs would improve micro-channel definition.
3. **Network Dependency for Dynamic Rainfall:** Rainfall queries rely on external Open-Meteo APIs. While the regional fallback ensures uninterrupted service, an embedded offline precipitation lookup table would further harden air-gapped deployments.

9. AI Tool Usage Declaration

In compliance with the assignment's LLM and AI Tool Usage Policy:

- **AI Tools Consulted:** Google Gemini 2.5 Flash and Antigravity Assistant.
- **Scope of AI Assistance:** AI assistance was utilized for boilerplate UI styling in `style.css`, reformatting and populating the LaTeX manuscript template from provided system specifications, drafting unit tests in `test_api.py`, and generating client-side drag bounding box calculation math in `app.js`.
- **Verification and Ownership:** All core algorithms (stream KML extraction, UTM metric transformation, Priority-Flood sink filling, D8 steepest-descent flow direction routing, reverse queue catchment delineation, Rational runoff estimation, and supervisor daemon shell scripts) were conceptualized, reviewed, debugged, and verified by the student.

Appendix A. Source Code, Video Demonstration & Deployment

The complete, version-controlled source code, video demonstration, and deployment for this project are publicly available:

GitHub Repository: https://github.com/kcharithreddy/Contour_Map_Planning

Video Demonstration Playlist: https://www.youtube.com/playlist?list=PLAHMqE-Hy_PY

Live Deployment (Lab VM): <http://10.1.75.51:3245/>

A.1 Project Directory Structure

```

contourmao/
├── app/
│   ├── __init__.py
│   ├── dem.py           # Coordinate reprojection (UTM Zone 44N) & DEM grid construction
│   ├── hydrology.py     # Rational Runoff equation (V = P x A x C) & pond dimensioning
│   ├── main.py          # FastAPI endpoints, boundary clamping & selection limit validation
│   ├── parser.py        # lxml stream parsing of KML/KMZ & bounding box polyline filtering
│   ├── pond.py          # Optimal pond site selection & reverse flow catchment delineation
│   ├── schemas.py       # Pydantic v2 data models & JSON API contracts
│   ├── terrain.py       # Priority-Flood sink filling & D8 flow direction/accumulation
│   └── static/
│       ├── app.js       # Leaflet GIS interaction, drag box HUD & GeoJSON rendering
│       ├── index.html   # Full-screen responsive dashboard UI
│       └── style.css     # Glassmorphism styling, ripple animations & KPI cards
├── tests/
│   ├── test_api.py      # API endpoints, bounding checks & minimum area limit tests
│   ├── test_dem.py      # DEM generation & auto-coarsening memory tests
│   ├── test_golden.py   # Regression golden benchmarks against sample survey file
│   ├── test_parser.py   # KML stream parsing & error handling tests
│   ├── test_pond.py     # Catchment delineation & outlet selection tests
│   └── test_terrain.py  # Depression filling & D8 slope calculation tests
├── figures/
├── contours_1m (1).kml  # Embedded high-resolution website and catchment screenshots
├── contours_1m (1).kml  # Reference village 3D contour survey dataset (6.7 MB)
├── deploy.py           # Automated remote SSH/SFTP deployment & smoke-test script
├── requirements.txt    # Minimal pure-Python dependency specifications
├── start_daemon.sh     # Multi-port (3245/3000) supervisor daemon script
└── PHASE3_SUBMISSION_GUIDE.md # 5-minute video presentation script & submission guide

```

A.2 Deployment Instructions

```

# 1. Local Execution:
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python3 -m uvicorn app.main:app --host 0.0.0.0 --port 3245 # Access http://localhost:3245

# 2. Remote Multi-Port Supervisor Daemon Execution:
chmod +x start_daemon.sh && setsid ./start_daemon.sh /dev/null 2>&1 &
# Running live on http://10.1.75.51:3245 and http://10.1.75.51:3000

# 3. Automated Test Verification Suite:
pytest tests/ -v # Validates all 33 unit and golden tests

```